SYSTEM ARCHITECTURE & ENGINEERING SPECIFICATION

MusicSphere — Distributed Mobile Audio Sync Engine

Serverless Peer-to-Peer Synchronized Playback Blueprint (Android Native MVP + Cross-Platform Protocol)

ARCHITECTURE	PRIMARY STACK	SYNC TOLERANCE	ROOM CONTROL
100% Serverless P2P	**Kotlin + Compose + C++ Oboe**	**≤ 2.0 ms (Acoustic Phase Lock)**	**Collaborative Party Queue**

1. EXECUTIVE OVERVIEW & ARCHITECTURAL DECISIONS

MusicSphere enables multiple mobile devices in the same physical space to play music out loud in tight acoustic synchronization over local Wi-Fi or a Mobile Hotspot without requiring any cloud backend. The Host device operates an embedded WebSocket control server alongside a high-precision UDP master clock and audio broadcaster.

Mode A: In-App YouTube / YT Music (Primary)

Because all connected client devices have internet access, tracks queued inside MusicSphere's embedded player do not require streaming heavy raw PCM bytes over local Wi-Fi. The Host broadcasts the `videoId` and a scheduled `playAtMasterTimeMs` timestamp via WebSocket. Every phone streams the audio directly and runs a 3-tier Phase-Locked Loop (PLL) adjusting HTML5 `playbackRate` (`0.97×`–`1.03×`) to maintain sub-10ms alignment.

Mode B: External System Audio Broadcast (Fallback)

When the Host toggles on external system capture, MusicSphere intercepts raw 48 kHz 16-bit Stereo PCM 1s and 0s via Android's `MediaProjection` and `AudioPlaybackCapture` API. A native C++ JNI bridge compresses 20ms chunks into 320-byte Opus frames, stamps them with a future execution timestamp, and unicasts them over UDP port 5004 to all peers for Oboe low-latency rendering.

2. PSYCHOACOUSTIC DESYNCHRONIZATION THRESHOLDS & ENGINEERING TARGETS

When multiple mobile speakers play identical audio in the same room, acoustic wave summation dictates strict timing tolerances. The engineering target for MusicSphere is to keep inter-device playback error within **±1.5 ms**:

Desync Offset (Δt)	Acoustic Phenomenon	Perceptual Result in Room	System Mitigation Strategy
0.0 to 1.5 ms	Phase Alignment	Tight, unified stereo/multi-speaker soundstage.	Target state; lock playback rate at `1.00×`.
1.5 to 10.0 ms	Comb Filtering	Hollow, metallic, or robotic "flanging" cancellation.	Engage micro-resampling / Acoustic Chirp Calibration.
10.0 to 30.0 ms	Haas (Precedence) Effect	Sound localizes solely to the earliest phone; muddy transients.	Dynamic PLL drift correction (`0.97×` / `1.03×`).
> 30.0 ms	Slapback Echo	Distinct stadium echo; unendurable group listening.	Hard buffer seek / RingBuffer frame drop or zero-stuffing.

3. END-TO-END DUAL-ENGINE SYSTEM ARCHITECTURE

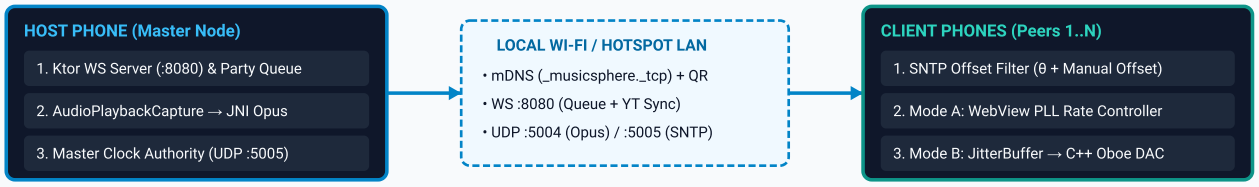


Figure 1: Serverless Local Wi-Fi / Hotspot Topology showing Control Plane (WS) and Real-Time Media Plane (UDP).

4. TECHNOLOGY STACK & CORE DEPENDENCIES

Subsystem	Technology / Library	Engineering Purpose
UI & State	Kotlin 2.0, Jetpack Compose (M3), Coroutines/Flow	Reactive room UI, collaborative queue list, real-time sync diagnostics.
Zero-Config Discovery	Android `NsdManager` (mDNS) + ZXing QR	Broadcasts `_musicsphere._tcp.`; QR fallback bypasses hotspot mDNS filters.
Control & Queue Server	Ktor CIO Embedded Server & WebSockets (`:8080`)	Runs on Host phone; manages collaborative queue additions, votes, and YT state.
Clock & Audio Transport	Java/Kotlin `DatagramSocket` (UDP `:5004`, `:5005`)	Multi-unicast UDP avoids 802.11 multicast base-rate throttling (1 Mbps bottleneck).
Native Audio & Codec	C++17 NDK, Google Oboe (AAudio), `libopus`	Hardware low-latency DAC output and 20ms frame encoding at 128 kbps.
System Loopback	`MediaProjection` + `AudioPlaybackCapture`	Captures internal PCM audio from external apps permitting playback capture.

5. REPOSITORY & MODULE STRUCTURE

```
musicsphere-android/ ├── app/src/main/ │ ├── cpp/ # C++17 NDK Low-Latency Audio & DSP Engine │ │ ├── CMakeLists.txt # Pulls Google Oboe & Xiph libopus via FetchContent │ │ ├── jni_bridge.cpp # JNI bindings exposed to NativeAudioBridge.kt │ │ ├── AudioEngine.h / .cpp # Oboe AudioStreamDataCallback & timestamp matching │ │ ├── OpusCodec.h / .cpp # 48kHz Stereo 20ms frame encoder/decoder │ │ ├── JitterBuffer.h / .cpp # Lock-free priority queue ordered by playAtMasterUs │ │ └── AcousticCalibrator.h / .cpp # Cross-correlation FM chirp detector for auto-sync │ └── java/com/musicsphere/app/ │ ├── data/model/ # QueueItem.kt, RoomState.kt, SyncPacket.kt │ ├── network/nsd/NsdDiscoveryManager.kt # mDNS service advertiser and discovery scanner │ ├── network/ws/HostWebSocketServer.kt # Ktor CIO embedded WebSocket server (Party Queue) │ ├── network/ws/ClientWebSocketManager.kt # Ktor WebSocket client & JSON event dispatcher │ ├── network/udp/ClockSyncEngine.kt # Port 5005 8-burst SNTP median clock synchronizer │ ├── network/udp/UdpAudioTransport.kt # Port 5004 multi-unicast Opus packet broadcaster │ ├── audio/service/MediaCaptureService.kt # FOREGROUND_SERVICE_TYPE_MEDIA_PROJECTION │ ├── audio/capture/SystemAudioCapturer.kt # AudioRecord PCM reader feeding JNI Opus encoder │ ├── audio/youtube/YouTubeSyncPlayer.kt # WebView IFrame player + JS playbackRate PLL loop │ └── ui/ # HomeScreen.kt, RoomScreen.kt, SyncModal.kt
```

6. MATHEMATICAL CLOCK SYNCHRONIZATION PROTOCOL (UDP PORT 5005)

Hardware system clocks (`SystemClock.elapsedRealtimeNanos()`) differ across phones and drift over time due to crystal oscillator skew. Every 2.0 seconds, each Client dispatches a burst of 8 UDP probe packets (spaced 25ms apart) to the Host on port 5005. Using monotonic microsecond timestamps t_1 (Client send), t_2 (Host receive), t_3 (Host reply), and t_4 (Client receive):

Round-Trip Network Delay (δ): $\delta = (t_4 - t_1) - (t_3 - t_2)$ | **Clock Offset (θ):** $\theta = [(t_2 - t_1) + (t_3 - t_4)] \div 2$

Master Clock Projection: $T_{master} = T_{client_monotonic} + \theta_{EMA} + \Delta T_{manual_calibration}$

Outlier Rejection Filter: From each 8-packet burst, the Client discards the 4 samples with the highest round-trip delay δ (eliminating Wi-Fi CSMA/CA contention spikes), computes the median of the remaining 4 θ values, and applies an Exponential Moving Average ($\alpha = 0.20$) to update θ_{EMA} smoothly without audio clicks.

7. MODE A SPECIFICATION: IN-APP YOUTUBE COLLABORATIVE QUEUE & PLL SYNC

WebView JavaScript Bridge & 3-Tier Phase-Locked Loop (PLL)

Whenever a track starts, seeks, or every 3 seconds during playback, the Host broadcasts a SYNC_STATE JSON frame over WebSocket containing `videoId`, `trackPositionMs`, and `atMasterClockMs`. Every 250ms, each Client evaluates its instantaneous drift error: $E_{drift} = P_{actual_webview} - [P_{host_track} + (T_{master} - T_{at_master})]$.

- **Tier 1 — Hard Seek ($|E_{drift}| > 150$ ms):** Executes `player.seekTo(expectedSec, true)` and resets rate to `1.00×`.
- **Tier 2 — Proportional Pitch-Safe Pull-In (8 ms < $|E_{drift}| \le 150$ ms):** Sets HTML5 `video.playbackRate = 1.03` if lagging ($E_{drift} < -8$ ms) or `0.97` if ahead ($E_{drift} > +8$ ms). A ±3% tempo shift is virtually imperceptible but corrects a 30ms error in 1 second.
- **Tier 3 — Phase Lock ($|E_{drift}| \le 8$ ms):** Restores `video.playbackRate = 1.00`.

8. MODE B SPECIFICATION: BINARY UDP AUDIO PACKET SCHEMA (PORT 5004)

In External App Broadcast mode, `SystemAudioCapturer` reads 20ms stereo PCM frames (960 samples/channel at 48 kHz = 3,840 bytes), compresses them via JNI `libopus` to ~320 bytes, stamps a scheduled playback time ($T_{play} = T_{capture} + 120{,}000\ \mu s$), and transmits this 16-byte binary header + payload to each peer IP:

Byte Offset	Field Name	Data Type	Description & Validation Rule
`0..1`	`magic`	`uint16_t` (BE)	Protocol identifier 0x4D53 ("MS"). Drop packet if mismatch.
`2..5`	`seqNumber`	`uint32_t` (BE)	Monotonically increasing frame index used by Opus PLC for lost packets.
`6..13`	`playAtMasterUs`	`int64_t` (BE)	Exact Master Clock timestamp (in µs) when frame must exit the DAC.
`14..15`	`payloadLen`	`uint16_t` (BE)	Byte length of the compressed Opus frame (typically 280–340 bytes).
`16..N`	`opusPayload`	`uint8_t[]`	Raw Opus-encoded stereo audio frame (20ms duration).

9. HARDWARE DAC LATENCY COMPENSATION & CALIBRATION ENGINE

1. Manual Sync Nudge Slider (-150 to +150 ms)

Because the Host phone plays system audio immediately at $T = 0$ ms in External Broadcast Mode while Client phones buffer for 120 ms, and different Android OEMs exhibit varying HAL audio latencies (15 ms on Pixel vs. 65 ms on budget devices), every client screen exposes a persistent **1 ms precision Sync Nudge Slider**. Adjustments shift $\Delta T_{manual_calibration}$ immediately and persist per device hardware model in Jetpack `DataStore`.

2. One-Tap Acoustic Chirp Auto-Calibration

When triggered, the Client opens a 48 kHz mono `AudioRecord` mic stream for 1.5 seconds and sends `REQ_CHIRP` to the Host. The Host plays a 15ms linear FM sweep ($2\ kHz \rightarrow 6\ kHz$) at T_0 , and the Client emits the identical sweep at $T_0 + 250\ ms$. `AcousticCalibrator.cpp` runs cross-correlation against the reference template to detect the exact acoustic peak separation and auto-configures $\Delta T_{manual_calibration}$.

10. PHASED IMPLEMENTATION MILESTONES (EXECUTION ROADMAP)

Milestone	Core Deliverables & Files	Verification & Acceptance Criteria
Phase 1: Discovery & P2P Handshake	`build.gradle.kts`, `NsdDiscoveryManager.kt`, QR Code generator/scanner, `HomeScreen.kt`.	Two phones on the same Wi-Fi or Mobile Hotspot discover each other via mDNS or QR scan in < 2 seconds.
Phase 2: Ktor Party Queue Server	`HostWebSocketServer.kt`, `ClientWebSocketManager.kt`, `QueueItem.kt`, `RoomScreen.kt`.	Any connected peer can search YouTube tracks, append to the shared room queue, and see live state updates.
Phase 3: SNTP Clock & Mode A Sync	`ClockSyncEngine.kt` (UDP `:5005`), `YouTubeSyncPlayer.kt` with WebView JS PLL loop.	Clock offset θ converges within 4 seconds; two phones play the same YouTube track with < 10ms drift.
Phase 4: Mode B Native C++ & Opus	`CMakeLists.txt`, `OpusCodec.cpp`, `AudioEngine.cpp`, `MediaCaptureService.kt`, UDP `:5004`.	External app audio is captured via `MediaProjection`, Opus-encoded, transmitted over UDP, and played via Oboe.
Phase 5: Acoustic Auto-Calibration	`AcousticCalibrator.cpp`, `SyncModal.kt`, persistent OEM offset storage in `DataStore`.	Manual slider shifts audio phase smoothly; acoustic beep test detects speaker delay within ±2 ms accuracy.

11. COPY-PASTE MASTER PROMPT FOR YOUR CODING AGENT

Agent Initialization Prompt (Paste into Cursor / Claude Code / Windsurf)

"You are a Staff Android & Real-Time Audio Systems Engineer. We are building **MusicSphere**, a serverless peer-to-peer synchronized group music app in Kotlin, Jetpack Compose, Ktor CIO Embedded WebSockets, and C++17 NDK (Google Oboe + libopus). Follow the 5-Phase Architecture Specification strictly: (1) Configure `minSdk = 29`, Kotlin 2.0, Ktor Server/Client CIO, ZXing, and CMake with `FetchContent` for Oboe and Opus; (2) Implement `NsdDiscoveryManager` (`_musicsphere._tcp.`) and QR code IP pairing; (3) Build `HostWebSocketServer` on port 8080 for the collaborative party queue; (4) Build `ClockSyncEngine` on UDP port 5005 using an 8-probe SNTP burst with median outlier rejection and EMA smoothing, paired with `YouTubeSyncPlayer` using a 3-tier HTML5 `playbackRate` PLL (`0.97×–1.03×`); (5) Implement `MediaCaptureService` using `AudioPlaybackCaptureConfiguration`, JNI Opus 20ms frame encoding, UDP port 5004 multi-unicast transport, and Oboe timestamp-scheduled playback with a ±150ms calibration slider. Start by generating Milestone 1 and Milestone 2 completely with zero placeholders."